

Campus safety escort matching functions ▾

hey , i want functions in python , no ui , no unit tests , give me the python funtions only. this is the app descriptionThe university is concerned about student safety on campus, especially during evening hours. Some students feel uneasy walking alone between buildings or across the campus. To address this, the university is planning a mobile application that connects students who need someone to accompany them with other students willing to help.

Imagine a student leaving the library late at night who wants to walk safely to the dorms. They can submit a request through the app, indicating where they are, where they want to go, and the time they need help. On the other side, students volunteering as “helpers” can see requests and respond to them.

Since many students may request assistance at similar times, the system needs to manage multiple incoming requests while ensuring that helpers are not overwhelmed or assigned in conflicting ways. In some situations, there may be no helpers available at all, while in others, several helpers may respond to the same request.

The system must ensure that helpers receive only the information necessary to assist the student safely, without exposing sensitive details about the student. At the same time, helpers should be able to communicate with students quickly, safely, and reliably. The university expects that the system remains responsive even when many students are using it at the same time.

Show less

May 27 ↺ ✎ 📄

Check available skills >

Here are the Python functions for the campus safety walk companion app, organized by domain:

Created a file, read a file >

Here's a breakdown of what's included, organized by concern:

Data models — `WalkRequest`, `Helper`, `Message` dataclasses plus `RequestStatus` / `HelperStatus` enums to represent all core entities.

Privacy layer

- `generate_anonymous_token` — hashes the student ID + request ID into a one-time token; helpers never see real student IDs

- `sanitize_location` — strips emails, phone numbers, and student ID patterns from free-text location inputs
- `build_helper_visible_request` — whitelist-based projection that returns only the fields a helper is permitted to see

Request lifecycle — `create_walk_request`, `cancel_walk_request`, `expire_stale_requests`, `complete_walk_request`

Helper management — `register_helper`, `set_helper_availability`, `get_available_helpers`, `update_helper_rating` (rolling average)

Matching engine

- `match_helper_to_request` — assigns the best-rated available helper to a pending request
- `handle_multiple_helper_responses` — gracefully handles the case where several helpers respond to the same request (picks best, returns the rejected list for notification)
- `release_helper` — frees a helper after a walk ends

Messaging — `send_message`, `get_conversation`, `send_system_notification` — all messages use anonymous tokens, never real IDs

Concurrency — `RequestQueue` (thread-safe priority queue with a lock), `process_request_worker` (background loop), `start_request_processor` (spawns the daemon thread + returns a stop event for clean shutdown)

Admin/reporting — `get_requests_by_status`, `get_helper_history`, `summarize_activity` (aggregate counts, no PII)

 Campus safety
PY

Download

